

MIT 6.S191: Introduction to Deep Learning

Lecture 1 — Quick Reference Notes

Tayaba Abbasi

September 8, 2026

These notes were compiled while studying MIT 6.S191, *Introduction to Deep Learning*, and from personal lecture notes. They are intended as a concise revision and interview-preparation reference rather than a replacement for the official lecture.

Contents

1	What Is Deep Learning?	3
1.1	Hierarchical Feature Learning	3
2	Why Deep Learning, and Why Now?	3
2.1	Big Data	3
2.2	Hardware	3
2.3	Software	4
3	The Perceptron	4
3.1	Role of the Bias	4
3.2	Forward Propagation	4
4	Activation Functions	5
4.1	Sigmoid	5
4.2	Hyperbolic Tangent (tanh)	5
4.3	Rectified Linear Unit (ReLU)	5
5	From Perceptrons to Neural Networks	6
5.1	Multiple Outputs and Dense Layers	6
5.2	Single Hidden Layer	6
5.3	Deep Neural Networks	6
6	Loss Functions	6

6.1	Empirical Loss	7
6.2	Binary Cross-Entropy	7
6.3	Mean Squared Error	7
7	Training as an Optimization Problem	7
8	Gradient Descent	8
8.1	Basic Gradient-Descent Algorithm	8
9	Backpropagation	8
10	The Learning Rate	9
10.1	Adaptive Learning Rates	9
11	Stochastic Gradient Descent and Mini-Batches	9
11.1	Stochastic Gradient Descent	9
11.2	Mini-Batch Gradient Descent	9
12	Overfitting and Generalization	10
13	Regularization	10
13.1	Dropout	10
13.2	Early Stopping	11
14	Lecture 1: Compact Mental Model	11
15	Interview Review Questions	11

1 What Is Deep Learning?

Artificial intelligence, machine learning, and deep learning can be viewed as nested ideas:

- **Artificial Intelligence (AI):** algorithms and systems designed to process information and support intelligent decisions or behaviour.
- **Machine Learning (ML):** a subset of AI in which systems learn patterns from data instead of being explicitly programmed with every rule.
- **Deep Learning (DL):** a subset of machine learning that learns representations directly from data using neural networks with multiple layers.

A central idea in deep learning is to learn a task *directly from raw data* rather than relying entirely on manually designed features.

Key Idea

Traditional machine-learning pipelines often depend heavily on human-designed features. Deep learning instead learns useful representations automatically and hierarchically from data.

1.1 Hierarchical Feature Learning

Deep networks can build representations from low-level to high-level structure. In a vision example:

- **Low-level features:** lines and edges,
- **Mid-level features:** eyes, noses, ears, shapes,
- **High-level features:** complete facial structure or semantic objects.

The hierarchy is learned from data rather than manually specified feature by feature.

2 Why Deep Learning, and Why Now?

Many of the core ideas behind neural networks are decades old. Their modern success is strongly connected to three developments.

2.1 Big Data

Larger datasets, easier data collection, and improved storage provide the examples needed to train high-capacity models.

2.2 Hardware

Graphics Processing Units (GPUs) enable highly parallel computation, substantially accelerating neural-network training.

2.3 Software

Modern software frameworks and tooling make neural networks easier to build, train, test, and deploy. Common examples include TensorFlow and PyTorch.

Key Idea

The recent dominance of deep learning is not because neural networks are brand new; it is because data, compute, and software have made them practical at scale.

3 The Perceptron

The **perceptron** is a fundamental computational building block of neural networks.

Suppose a neuron receives m inputs:

$$\mathbf{x} = [x_1, x_2, \dots, x_m]^T$$

with corresponding weights

$$\mathbf{w} = [w_1, w_2, \dots, w_m]^T.$$

The neuron first computes a linear combination:

$$z = w_0 + \sum_{i=1}^m x_i w_i,$$

where w_0 is the **bias**. In vector form,

$$z = w_0 + \mathbf{x}^T \mathbf{w}.$$

The output is then obtained by applying an activation function g :

$$\hat{y} = g(z) = g(w_0 + \mathbf{x}^T \mathbf{w}).$$

3.1 Role of the Bias

The bias gives the neuron an additional learnable degree of freedom and allows the activation or decision boundary to shift rather than being constrained to pass through the origin.

3.2 Forward Propagation

Forward propagation through a neuron can be summarized as:

Inputs $\rightarrow$ Weighted Sum + Bias $\rightarrow$ Non-linearity $\rightarrow$ Output

or simply,

$$z = \mathbf{x}^T \mathbf{w} + b, \quad \hat{y} = g(z).$$

4 Activation Functions

Activation functions introduce **non-linearity** into neural networks. Without non-linear activation functions, stacking multiple linear layers would still produce only a linear transformation.

Non-linearities allow neural networks to approximate much more complex functions and decision boundaries.

4.1 Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

with derivative

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

The sigmoid maps real-valued inputs to the interval $(0, 1)$ and is therefore useful when an output is interpreted as a probability.

4.2 Hyperbolic Tangent (tanh)

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

with derivative

$$\frac{d}{dz} \tanh(z) = 1 - \tanh^2(z).$$

Its output lies between -1 and 1 .

4.3 Rectified Linear Unit (ReLU)

$$\text{ReLU}(z) = \max(0, z).$$

Its derivative is

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & \text{otherwise.} \end{cases}$$

ReLU constrains negative activations to zero while leaving positive values unchanged.

Interview Check

Q: Why do neural networks need non-linear activation functions?

Because a composition of purely linear transformations is still linear. Non-linear activations allow a network to represent complex, non-linear relationships and decision boundaries.

5 From Perceptrons to Neural Networks

5.1 Multiple Outputs and Dense Layers

A layer can contain multiple neurons. If every input is connected to every output neuron, the layer is called a **dense** or **fully connected** layer.

For output neuron i ,

$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}, \quad y_i = g(z_i).$$

Each neuron has its own set of weights and bias.

5.2 Single Hidden Layer

A neural network can map an input vector into a hidden representation and then transform that representation into the final output:

$$\mathbf{x} \rightarrow \mathbf{z}^{(1)} \rightarrow \mathbf{h}^{(1)} \rightarrow \hat{\mathbf{y}}.$$

The first weight matrix transforms the input into the hidden space, and a second weight matrix transforms the hidden representation into the output space.

5.3 Deep Neural Networks

A deep neural network stacks several layers, typically placing non-linear activation functions between linear transformations:

$$\mathbf{x} \rightarrow \text{Layer 1} \rightarrow \text{Layer 2} \rightarrow \dots \rightarrow \text{Layer } L \rightarrow \hat{\mathbf{y}}.$$

The final prediction is obtained by repeated forward propagation through the layers.

Listing 1: Conceptual PyTorch-style dense network

```
model = nn.Sequential(
    nn.Linear(input_dim, hidden_dim),
    nn.ReLU(),
    nn.Linear(hidden_dim, output_dim)
)
```

6 Loss Functions

Training requires a way to quantify how different a prediction is from the ground-truth target.

For one training example, the loss can be written as

$$\mathcal{L}(f(\mathbf{x}^{(i)}; \mathbf{W}), y^{(i)}),$$

where $f(\mathbf{x}^{(i)}; \mathbf{W})$ is the model prediction and $y^{(i)}$ is the true target.

A smaller loss indicates a prediction closer to the desired output; a larger loss indicates a greater error.

6.1 Empirical Loss

Training is normally concerned with performance over a dataset rather than a single example. The empirical loss is

$$J(\mathbf{W}) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(\mathbf{x}^{(i)}; \mathbf{W}), y^{(i)}).$$

This quantity is also commonly described as an objective function, cost function, or empirical risk.

6.2 Binary Cross-Entropy

For binary classification with probabilistic outputs,

$$J(\mathbf{W}) = -\frac{1}{n} \sum_{i=1}^n \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right].$$

6.3 Mean Squared Error

For regression problems with continuous outputs,

$$J(\mathbf{W}) = \frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - \hat{y}^{(i)} \right)^2.$$

Interview Check

Q: What is the difference between a loss function and an optimizer?

The loss function defines *what* error is being minimized. The optimizer defines *how* the model parameters are updated to reduce that loss.

7 Training as an Optimization Problem

The objective of training is to find the network parameters that minimize empirical loss:

$$\mathbf{W}^* = \arg \min_{\mathbf{W}} J(\mathbf{W}).$$

Here, $\mathbf{W}$ represents the collection of weights across all layers of the network.

8 Gradient Descent

Gradient descent iteratively updates the parameters in the direction that decreases the loss.

The gradient

$$\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$$

provides local information about how the loss changes with respect to the weights.

The standard update rule is

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$$

where η is the **learning rate**.

8.1 Basic Gradient-Descent Algorithm

1. Initialize weights randomly.
2. Compute the loss $J(\mathbf{W})$.
3. Compute the gradient $\partial J / \partial \mathbf{W}$.
4. Take a small step in the negative-gradient direction.
5. Repeat until convergence or until a stopping criterion is met.

Neural-network loss landscapes are generally complex and non-convex, so optimization behaviour can depend on initialization, learning rate, architecture, and regularization.

9 Backpropagation

Backpropagation is the algorithm used to efficiently compute gradients of the loss with respect to the network's parameters.

It applies the chain rule from the output layer backward through the network.

For a simple chain

$$x \rightarrow z_1 \rightarrow \hat{y} \rightarrow J,$$

the derivative with respect to a later weight w_2 can be written as

$$\frac{\partial J}{\partial w_2} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_2}.$$

For an earlier weight w_1 , the chain becomes longer:

$$\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_1} \frac{\partial z_1}{\partial w_1}.$$

This process is repeated for the weights throughout the network, reusing gradients from later layers.

Interview Check

Q: What is the difference between backpropagation and gradient descent?

Backpropagation computes the gradients of the loss with respect to model parameters.

Gradient descent uses those gradients to update the parameters.

10 The Learning Rate

The learning rate η controls the step size taken during parameter updates.

- A **very small learning rate** can make convergence slow and may leave optimization stuck in undesirable regions.
- A **very large learning rate** can overshoot useful minima, become unstable, or cause the loss to diverge.
- A **well-chosen learning rate** gives smoother and more stable convergence.

10.1 Adaptive Learning Rates

Rather than keeping the learning rate fixed, adaptive optimization methods can vary the effective step size based on properties such as gradient magnitude, learning dynamics, and parameter scale.

Key Idea

The learning rate is one of the most influential hyperparameters in neural-network training because it directly controls how aggressively the optimizer changes the weights.

11 Stochastic Gradient Descent and Mini-Batches

Computing an exact gradient over an entire large dataset can be expensive.

11.1 Stochastic Gradient Descent

In pure stochastic gradient descent (SGD), a single randomly selected training example is used to estimate the gradient:

$$\frac{\partial J_i(\mathbf{W})}{\partial \mathbf{W}}.$$

This estimate is inexpensive but noisy.

11.2 Mini-Batch Gradient Descent

A common middle ground is to use a mini-batch of B examples:

$$\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} \approx \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(\mathbf{W})}{\partial \mathbf{W}}.$$

Mini-batches provide:

- a more accurate estimate of the gradient than a single example,
- smoother convergence,
- efficient parallel computation,
- substantial speedups on GPUs.

12 Overfitting and Generalization

Overfitting occurs when a model fits the training data too closely and fails to generalize well to unseen examples.

At the other extreme, **underfitting** occurs when a model does not have enough capacity, training, or appropriate structure to learn the patterns in the data.

The goal is not simply to minimize training error, but to achieve strong performance on unseen data.

13 Regularization

Regularization constrains the optimization problem in ways that discourage overly complex solutions and improve generalization.

13.1 Dropout

During training, dropout randomly sets a fraction of activations to zero. This prevents the network from depending too heavily on any single pathway or neuron.

Conceptually,

$$h'_j = m_j h_j, \quad m_j \sim \text{Bernoulli}(1 - p),$$

where p is the dropout probability.

The dropout mask is resampled during training, so different subsets of activations are removed on different passes.

Listing 2: Dropout examples

```
# TensorFlow / Keras
layer = tf.keras.layers.Dropout(rate=0.5)

# PyTorch
layer = nn.Dropout(p=0.5)
```

13.2 Early Stopping

Early stopping monitors performance on held-out data and stops training before continued optimization begins to worsen generalization.

A typical pattern is:

- training loss continues to decrease,
- validation/test loss stops improving or begins to increase,
- training is stopped around the point of best held-out performance.

Interview Check

Q: How does dropout help reduce overfitting?

Dropout randomly removes activations during training, forcing the network to distribute useful information across multiple pathways instead of relying too strongly on individual neurons.

14 Lecture 1: Compact Mental Model

The complete training pipeline can be summarized as:

Data $\rightarrow$ Forward Pass $\rightarrow$ Prediction $\rightarrow$ Loss $\rightarrow$ Backpropagation $\rightarrow$ Gradient Update

Repeated over many examples and iterations:

1. The network receives input data.
2. Each layer computes a weighted transformation followed by a non-linearity.
3. The final layer produces a prediction.
4. A loss function measures prediction error.
5. Backpropagation computes gradients using the chain rule.
6. An optimizer updates the weights.
7. Mini-batches make training computationally practical.
8. Regularization techniques such as dropout and early stopping help the model generalize.

15 Interview Review Questions

1. What is the relationship between AI, machine learning, and deep learning?
2. Why did deep learning become much more successful in recent years?
3. What does a perceptron compute?
4. What is the role of a bias term?
5. Why are non-linear activation functions necessary?
6. How do sigmoid, tanh, and ReLU differ?

7. What is a dense layer?
8. What makes a neural network “deep”?
9. What does a loss function measure?
10. When would you use cross-entropy versus mean squared error?
11. What is empirical loss?
12. What does gradient descent do?
13. What is the role of the learning rate?
14. What is backpropagation, and how does it use the chain rule?
15. What is the difference between backpropagation and optimization?
16. Why use SGD or mini-batches instead of the full dataset for every update?
17. What is overfitting?
18. How do dropout and early stopping improve generalization?

Source Note

These notes follow the content and sequence of MIT 6.S191 Lecture 1, *Introduction to Deep Learning*, taught by Alexander Amini, together with the author’s own lecture notes. Administrative material from the course slides has intentionally been omitted so that this document remains focused on technical concepts and interview revision.